

REST API CLIENT

SPIS TREŚCI

Spis treści.....	1
Cel zajęć.....	1
Rozpoczęcie.....	1
Uwaga.....	1
Wymagania.....	2
Badanie API.....	2
Implementacja.....	2
Commit projektu do GIT.....	4
Podsumowanie.....	4

CEL ZAJĘĆ

Celem głównym zajęć jest zdobycie następujących umiejętności:

- pobieranie danych z zewnętrznych zasobów za pomocą REST API
- zdobywanie wiedzy na temat zewnętrznych API za pomocą dokumentacji typu Swagger
- wysyłanie asynchronicznych żądań z wykorzystaniem XMLHttpRequest i Fetch API

W praktycznym wymiarze uczestnicy stworzą dynamiczną stronę HTML pozwalającą na wyświetlanie bieżącej informacji pogodowej oraz prognoz dla zadanej przez użytkownika miejscowości.

ROZPOCZĘCIE

Rozpoczęcie zajęć. Powtórzenie wykonywania połączeń synchronicznych i asynchronicznych z poziomu JS na stronie.

Wejściówka?

UWAGA

Ten dokument aktywnie wykorzystuje niestandardowe właściwości. Podobnie jak w LAB A wejdź do Plik -> Informacje -> Właściwości -> Właściwości zaawansowane -> Niestandardowe i zaktualizuj pola. Następnie uruchom ten dokument ponownie lub Ctrl+A -> F9.

WYMAGANIA

W ramach LAB D przygotowane powinny zostać:

- pojedyncza strona HTML ze skryptem ładowanym z zewnętrznego pliku JS
- pole tekstowe (input typu „text”) do wprowadzania adresu
- przycisk „Pogoda”, po kliknięciu którego wykonywane jest zapytanie asynchroniczne:
 - do API Current Weather: <https://openweathermap.org/current> za pomocą XMLHttpRequest
 - do API 5 day forecast: <https://openweathermap.org/forecast5> za pomocą Fetch API
- obsługa zwrotki z obu API – wypisanie pogody bieżącej oraz prognoz poniżej pola wyszukiwania.

Wygeneruj klucz do API. Ponieważ aktywacja może chwilę potrwać, na czas trwania laboratorium możesz wykorzystać „służbowy” klucz: `7ded80d91f2b280ec979100cc8bbba94`. **UWAGA!** Klucz zostanie dezaktywowany niedługo po zajęciach. Musisz wygenerować swój własny.

W przypadku blokady twórczej można posiłkować się filmem: <https://www.youtube.com/watch?v=WoKp2qDFxKk> jednakże spróbuj rozwiązać ten problem samodzielnie!

Prowadzący omówi powyższe wymagania. Upewnij się, czy wszystko rozumiesz.

Tu umieść swoje notatki:

...notatki...

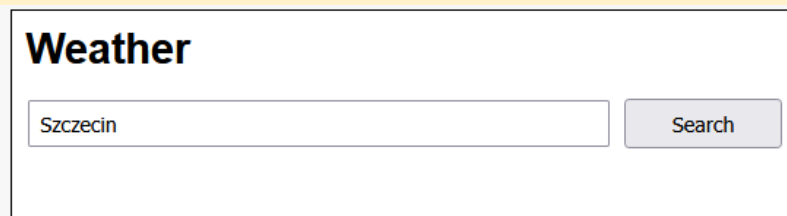
BADANIE API

Poświęć kilka minut na wykonanie przykładowych zapytań do API z poziomu pasku adresu przeglądarki. Podaj wymagane parametry dla osiągnięcia różnych wyników. Zbadaj odpowiedzi API, aby uzyskać pełen obraz wymagań i możliwości API.

IMPLEMENTACJA

Tradycyjnie implementację należy zacząć od zbudowania w HTML + CSS wszystkich wymaganych elementów / placeholderów na te elementy. Następnie krok po kroku należy implementować poszczególne zachowania.

Wstaw zrzut ekranu zawierającego stronę ze wszystkimi elementami, tj. pole tekstowe, przycisk, miejsce do wyświetlenia pogody i prognozy:



The screenshot shows a web interface with the title "Weather" in a large, bold font. Below the title is a text input field containing the word "Szczecin". To the right of the input field is a button labeled "Search". The entire form is enclosed in a simple rectangular border.

Punkty:	0	1
---------	---	---

Wstaw zrzut ekranu kodu odpowiedzialnego za wysyłanie żądania do current za pomocą XMLHttpRequest:

```
getCurrentWeather(query) : Promise<unknown> {
  return new Promise( executor: resolve => {
    const q :string = encodeURIComponent(query);
    const url :string = "https://api.openweathermap.org/data/2.5/weather?q=" +
      q + "&appid=" + this.apiKey + "&units=metric&lang=pl";
    const xhr :XMLHttpRequest = new XMLHttpRequest();
    xhr.open( method: "GET", url);

    xhr.onload = function () :void {
      const data = JSON.parse(xhr.responseText);
      console.log("Current weather:", data);
      resolve(data);
    };
    xhr.send();
  });
}
```

Wstaw zrzut ekranu pokazujący otrzymaną odpowiedź za pomocą `console.log()` w przeglądarce.

```
Current weather: script.js:20:29
▼ Object { coord: {...}, weather: (1) [...], base: "stations",
main: {...}, visibility: 10000, wind: {...}, clouds: {...}, dt:
1764955648, sys: {...}, timezone: 3600, ... }
  base: "stations"
  clouds: Object { all: 100 }
  cod: 200
  coord: Object { lon: 14.553, lat: 53.4289 }
  dt: 1764955648
  id: 3083829
  main: Object { temp: 4.34, feels_like: 3.42, temp_min:
3.88, ... }
  name: "Szczecin"
  sys: Object { type: 2, id: 2034200, country: "PL", ... }
  timezone: 3600
  visibility: 10000
  weather: Array [ {...} ]
  wind: Object { speed: 1.34, deg: 157, gust: 3.13 }
  <prototype>: Object { ... }

Current weather: script.js:20:29
▶ Object { coord: {...}, weather: (1) [...], base: "stations",
main: {...}, visibility: 10000, wind: {...}, clouds: {...}, dt:
1764955705, sys: {...}, timezone: 3600, ... }

Forecast: script.js:35:21
▶ Object { cod: "200", message: 0, cnt: 40, list: (40) [...],
city: {...} }

Forecast: script.js:35:21
▶ Object { cod: "200", message: 0, cnt: 40, list: (40) [...],
city: {...} }
```

Punkty:

0

1

Wstaw zrzut ekranu kodu odpowiedzialnego za wysyłanie żądania do forecast za pomocą Fetch:

```
getForecast(query, callback) :void {
  const q :string = encodeURIComponent(query);
  const url :string = "https://api.openweathermap.org/data/2.5/forecast?q=" +
    q + "&appid=" + this.apiKey + "&units=metric&lang=pl";
  fetch(url) Promise<Response>
    .then(function (response :Response ) {
      return response.json();
    }) Promise<any>
    .then(function (data) :void {
      console.log("Forecast:", data);
      callback(data);
    });
}
```

Wstaw zrzut ekranu pokazujący otrzymaną odpowiedź za pomocą `console.log()` w przeglądarce.

```
Forecast:
▼ Object { cod: "200", message: 0, cnt: 40, list: (40) [...],
city: {...} }
  ▼ city: Object { id: 3083829, name: "Szczecin", country:
    "PL", ... }
    ▶ coord: Object { lat: 53.4289, lon: 14.553 }
      country: "PL"
      id: 3083829
      name: "Szczecin"
      population: 407811
      sunrise: 1764918018
      sunset: 1764945894
      timezone: 3600
    ▶ <prototype>: Object { ... }
    cnt: 40
    cod: "200"
    ▶ list: Array(40) [ {...}, {...}, {...}, ... ]
      message: 0
    ▶ <prototype>: Object { ... }
```

Punkty:

0

1

Wstaw zrzut ekranu przedstawiającego wizualizację prognoz pogody:

Weather

Szczecin, Poland

Search

5.12.2025 18:35:04



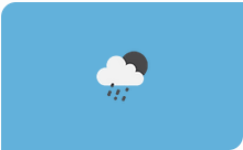
4 °C

Odczuwalna: 2 °C

zachmurzenie duże

Proгноza (co ~24h):

5.12.2025 19:00:00

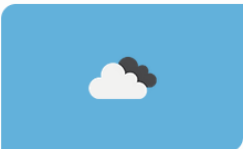


4 °C

Odczuwalna: 3 °C

slabe opady deszczu

6.12.2025 19:00:00



3 °C

Odczuwalna: 0 °C

zachmurzenie duże

7.12.2025 19:00:00



6 °C

Odczuwalna: 4 °C

zachmurzenie duże

Weather

Egipt

Search

5.12.2025 18:32:02



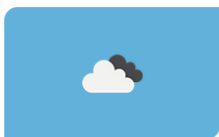
19 °C

Odczuwalna: 18 °C

zachmurzenie umiarkowane

Proгноza (co ~24h):

5.12.2025 19:00:00



19 °C

Odczuwalna: 18 °C

zachmurzenie umiarkowane

Inspector

Konsola

Debugger

Siec

...

Filtruj zawartosc

Bledy

Ostrzezenia

Informacje

Wpisy dziennika

Wpisy debugowania

CSS

XHR

Zadania

Current weather:

Object { coord: {}, weather: (1) [], base: "stations", main: {}, visibility: 10000, wind: {}, clouds: {}, dt: 1764956104, sys: {}, timezone: 3600, ... }

Current weather:

Object { coord: {}, weather: (1) [], base: "stations", main: {}, visibility: 10000, wind: {}, clouds: {}, dt: 1764956104, sys: {}, timezone: 3600, ... }

base: "stations"

clouds: Object { all: 100 }

cod: 200

coord: Object { lon: 14.553, lat: 53.4289 }

dt: 1764956104

id: 3083829

main: Object { temp: 4.34, feels_like: 2.36, temp_min: 3.88, ... }

name: "Szczecin"

sys: Object { type: 2, id: 2034200, country: "PL", ... }

timezone: 3600

visibility: 10000

weather: Array [{}]

wind: Object { speed: 2.24, deg: 166, gust: 3.58 }

<prototype>: Object { ... }

Forecast:

Object { cod: "200", message: 0, cnt: 40, list: (40) [], city: {} }

Forecast:

Object { cod: "200", message: 0, cnt: 40, list: (40) [], city: {} }

city: Object { id: 3083829, name: "Szczecin", country: "PL", ... }

cnt: 40

cod: "200"

list: Array(40) [{}, {}, {}, ...]

message: 0

<prototype>: Object { ... }

Inspector

Konsola

Debugger

Siec

...

Filtruj zawartosc

Bledy

Ostrzezenia

Informacje

Wpisy dziennika

Wpisy debugowania

CSS

XHR

Zadania

Current weather:

Object { coord: {}, weather: (1) [], base: "stations", main: {}, visibility: 10000, wind: {}, clouds: {}, dt: 1764955922, sys: {}, timezone: 7200, ... }

Current weather:

Object { coord: {}, weather: (1) [], base: "stations", main: {}, visibility: 10000, wind: {}, clouds: {}, dt: 1764955922, sys: {}, timezone: 7200, ... }

base: "stations"

clouds: Object { all: 51 }

cod: 200

coord: Object { lon: 30, lat: 27 }

dt: 1764955922

id: 357994

main: Object { temp: 19.43, feels_like: 18.34, temp_min: 19.43, ... }

name: "Egipt"

sys: Object { country: "EG", sunrise: 1764909307, sunset: 1764947188 }

timezone: 7200

visibility: 10000

weather: Array [{}]

wind: Object { speed: 3.94, deg: 339, gust: 4.85 }

<prototype>: Object { ... }

Forecast:

Object { cod: "200", message: 0, cnt: 40, list: (40) [], city: {} }

city: Object { id: 357994, name: "Egipt", country: "EG", ... }

cnt: 40

cod: "200"

list: Array(40) [{}, {}, {}, ...]

message: 0

<prototype>: Object { ... }

Upewnij się, że widoczne są pasek wyszukiwania ze wskazaną miejscowością, a także zarówno pogoda bieżąca jak i prognozy pogody.

Punkty:	0	1
---------	---	---

Strona 5 z 6

COMMIT PROJEKTU DO GIT

Zacommituj i pushnij swoje rozwiązanie do repozytorium GIT.

Upewnij się, czy wszystko dobrze się wysłało. Jeśli tak, to z poziomu przeglądarki utwórz branch o nazwie `lab-d` na podstawie głównej gałęzi kodu.

Podaj link do brancha `lab-d` w swoim repozytorium:

...link, np. <https://github.com/inazwisko/ai1-lab/tree/lab-d...>

<https://github.com/jaqbqlik/5-semestr-Aplikacje-Internetowe-/tree/main/lab-d>

PODSUMOWANIE

W kilku słowach/zdaniach napisz swoje przemyślenia odnośnie tego laboratorium. Nie używaj LLM.

...podsumowanie...

Zweryfikuj kompletność sprawozdania. Utwórz PDF i wyślij w terminie.